Bachelors Degree Project

A Behavioral Analysis of Data Exfiltration Techniques in Malicious Browser Extensions using Proxy-Based Monitoring



Author(s): Mohamed Shire
Mustafa Amged Mustafa
Supervisor(s): Mattias Davidsson
Semester: VT 2026
Course: 2DV50E
Subject: Computer Science

Abstract

Browser extensions have become a common part of modern web browsing, but they also introduce a significant security risk because they can access sensitive browser data and interact directly with web content. Malicious extensions can abuse these privileges to perform hidden data exfiltration, such as stealing cookies, logging keystrokes, or leaking browsing history. Detecting such behaviour is challenging for traditional endpoint protection methods, which often rely on host-based monitoring or static analysis. This thesis investigates whether malicious browser extension behaviour can instead be detected through out-of-band network traffic analysis.

Following the Design Science Research Methodology (DSRM), a scriptable proxy-based test environment was developed using mitmproxy to capture HTTP/1 and HTTP/2 traffic without modifying the browser execution environment. A benchmark dataset was created consisting of benign browser extensions and self-developed malicious extensions that simulated cookie theft, keystroke logging, and browsing history exfiltration in a stateful browser sandbox. Network traffic generated during execution was used to extract structural and behavioural features, which were then used to train and evaluate a Random Forest classifier with balanced class weights and Leave-One-Out Cross-Validation (LOOCV).

The results show that individual network requests were often difficult to classify accurately because malicious requests can resemble legitimate extension traffic. However, when suspicious requests were analysed in aggregate over time, detection performance improved substantially. In particular, applying a behavioural threshold of 5% or more suspicious requests made it possible to separate malicious traffic streams from benign ones while reducing false positives. The findings suggest that proxy-based network traffic analysis can serve as a lightweight and non-intrusive complement to existing extension security mechanisms for detecting post-installation malicious behaviour.

Keywords: Browser Extension Security, Network Traffic Analysis, Data Exfiltration

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Related work . . . 1
- 1.3 Problem formulation . . . 2
- 1.4 Motivation . . . 2
- 1.5 Results . . . 3
- 1.6 Scope/Limitation . . . 3
- 1.7 Target group . . . 3
- 1.8 Outline . . . 4

2 Method **4**
- 2.1 Research Project . . . 4
- 2.2 Research methods . . . 7
- 2.3 Network traffic analysis to detect malicious behaviour . . . 7
- 2.4 Dataset . . . 7
- 2.5 Evaluation . . . 8
- 2.6 Reliability and Validity . . . 8
- 2.7 Ethical Considerations . . . 9

3 Theoretical Background **10**
- 3.1 A Browser Extension Architecture and Privilege Models . . . 10
  - 3.1.1 The Manifest Blueprint and Permission Escalation . . . 10
  - 3.1.2 Content Scripts and Isolation Environments . . . 10
  - 3.1.3 Background Service Workers . . . 10
- 3.2 B Mechanics of Unauthorized Data Exfiltration . . . 10
  - 3.2.1 DOM Scraping and Event Interception Logic . . . 11
  - 3.2.2 Session Hijacking via Cookie Extraction . . . 11
- 3.3 C Proxy-Based Monitoring and SSL/TLS Inspection . . . 11
  - 3.3.1 Architectural Integration of Mitmproxy . . . 11
  - 3.3.2 SSL/TLS Termination and Upstream Certificate Sniffing . . . 11
  - 3.3.3 Event-Driven Inline Scripting Framework . . . 12
- 3.4 D Heuristic Detection and Entropy Analysis . . . 12
  - 3.4.1 Entropy Assessment . . . 12
  - 3.4.2 Temporal Correlation . . . 13

4 Research Project – Implementation **14**
- 4.1 System Architecture . . . 14
- 4.2 Malicious Extension Design . . . 14
- 4.3 Exfiltration Server . . . 15
- 4.4 Proxy Addon and Data Collection . . . 15
- 4.5 Data Collection Procedure . . . 15
- 4.6 Classifier Implementation . . . 15

5 Results **17**
- 5.1 Dataset . . . 17
- 5.2 Request-Level Classification . . . 17
- 5.3 Extension-Level Classification . . . 17
- 5.4 Feature Importances . . . 18

6 Analysis **19**
6.1 Request-Level Performance 19
6.2 Feature Importance 19
6.3 Extension-Level Classification and Threshold Analysis 19

7 Discussion **20**

8 Conclusions and Future Work **21**
8.1 Future Work 21

References **22**

1 Introduction

Web browsers are no longer only used to view websites, but are now central to everyday online activities such as communication, banking, shopping, and access to work-related systems. At the same time, browser extensions have become a common way for users to add new features and customize their browsing experience. However, because extensions can access website content and browser data, they can also pose a security risk if they are misused. Rather than focusing on browser permissions or extension review systems, this thesis examines the effectiveness of using a proxy together with network traffic analysis to detect data exfiltration behaviour performed by malicious browser extensions.

1.1 Background

Browser extensions often require extensive permissions in order to function as intended. Depending on their purpose, they may access cookies, browsing history, page content, or user input on websites. While this access is necessary for many legitimate extensions, it can also be abused by malicious or compromised extensions to collect sensitive information from the user. Examples of this include stealing cookies, gathering browsing history, or logging keystrokes and sending that data to an external server.

Current security mechanisms for web extensions mainly focus on the point of installation. Browsers and extension stores use permission systems, review processes, and static analysis to reduce the risk of malicious extensions being installed. However, these measures do not always address what happens after installation. An extension may appear harmless when it is reviewed, but later perform malicious actions through an update or by misusing permissions that were originally granted for legitimate reasons.

Because of this, it is relevant to study methods that focus on extension behaviour during use rather than only before installation. Previous research has shown that network traffic can be used as a useful source of information when analysing malicious software and browser extension behaviour, particularly when the goal is to identify suspicious outbound communication or signs of data exfiltration [1]. Since many malicious extension actions involve transmitting collected data outside the browser, this thesis investigates whether analysing extension-generated network traffic through a proxy can be used to detect such behaviour after installation.

1.2 Related work

Research on detection of malicious browser extensions has mostly focused on analyzing extension behaviour through static and dynamic techniques. Static analysis examines extension source code, permissions etc to identify possible security risks. While effective for detecting known patterns, these approaches are limited in capturing runtime behavior and may fail against obfuscation or dynamically triggered actions. Karim et al. [2] propose a static analysis approach to identify vulnerable browser extensions. The method analyzes extension source code by constructing function call graphs and extracting relevant program facts to detect capability leaks. While effective for identifying vulnerabilities, the approach is limited to static code analysis and may introduce performance overhead due to detailed tracking of program elements.

Dynamic analysis addresses these limitations by observing extensions during execution. Systems such as Arcanum apply dynamic taint tracking within Chromium-based browsers to trace the flow of privacy-sensitive data through extensions [1]. This enables fine-grained analysis of how data is accessed and transmitted. The deployment of such

systems has revealed significant privacy risks across thousands of extensions, highlighting the scale of the problem. However, these approaches require deep integration with the browser and access to internal execution context. Arcanum also does not interact with web pages or trigger extension behavior during experiments, meaning some data exfiltration cases may not be captured. This highlights a limitation in observing all possible malicious behaviors.

Hossain Shahriar et al. [3] proposed an HMM-based approach which monitors a broad set of features and combines both static and dynamic analysis of browser extensions. While this allows for more comprehensive detection, the implementation requires deep integration with the browser, making deployment complex and difficult to maintain as the browser evolves. In contrast, lightweight approaches that operate externally, such as proxy-based monitoring, avoid these challenges but may rely on more limited observable features.

1.3 Problem formulation

Current research can be split into two primary domains: static analysis which examine an extension's source code or files without executing it, in order to identify potentially malicious behaviour or suspicious patterns. Secondly, internal dynamic analysis, which requires researchers to modify the browser's source code (e.g. taint tracking in Chromium) to track exactly what the extension is doing second-by-second. While these dynamic approaches achieve high granularity, they may have some blindsides that can be exploited by malware authors.

Sophisticated malware is increasingly sandbox aware [4], capable of detecting the presences of internal loggers or taint-tracking hooks, thereby by pausing or surpressing malicious behavior to avoid being detected. Furthermore, as noted by Shahriar et al. [4], deep browser integration creates maintenance burden, as detection engines must be updated and re-engineered to match the evolution of browser engines.

Although network-based detection techniques are well established, there is limited research on how effectively browser extension data exfiltration can be detected using only externally observable HTTP traffic, without access to browser internals, extension source code, or runtime instrumentation.

To investigate this, this study addresses the following questions:

RQ1: To what extent can external network-layer features effectively distinguish between legitimate extension traffic and malicious data exfiltration?

RQ2: Which structural HTTP request features are most useful for distinguishing malicious exfiltration traffic from benign browser extension traffic?

1.4 Motivation

The problem addressed in this thesis is relevant from both a scientific and practical perspective. From a scientific viewpoint, much of the existing research on malicious browser extension detection focuses on static analysis or browser-internal dynamic analysis. While these approaches can provide detailed insight into extension behaviour, they often require access to extension code or modifications to the browser environment, making it more difficult to maintain. This thesis contributes by exploring a lightweight, external proxy-based approach that relies only on structural network traffic features. Evaluating the effectiveness of such an approach helps improve understanding of the trade-offs between visibility, interpretability, deployment complexity, and detection performance.

From a societal perspective, browser extensions are widely used and often have access to sensitive user data, including cookies and session information. Malicious or compromised extensions can exploit this access to leak personal data without the user's awareness. Improving the ability to detect such behavior contributes to better protection of user privacy and increases transparency around how data is handled in everyday web usage.

From an industrial perspective, lightweight and interpretable detection mechanisms are attractive because they are easier to deploy and maintain compared to more complex systems. A proxy-based approach can be integrated into existing network monitoring solutions without requiring significant changes to infrastructure. Understanding the effectiveness and limitations of such approaches can therefore support the development of more practical security solutions in real-world environments.

1.5 Results

This project produced three main results.

First, the request-level classifier achieved **97.69% accuracy** and a **91.30% F1 score** on 346 labelled requests using Leave-One-Out Cross-Validation (LOOCV). This shows that malicious extension traffic exhibits identifiable structural patterns.

Second, extension-level classification further improved practial reliability. By aggregating request predictions and applying a 5% threshold, the model achieved **100% recall** while eliminating all false positives, demonstrating the effectiveness of behavioural detection over time.

Third, feature importance analysis showed that `num_equals`, `avg_substring_length`, and `body_length` were the most influential features. These features reflect the structured nature of cookie and browsing-history exfiltration payloads.

All results were validated using LOOCV to maximise the available dataset and reduce overfitting.

1.6 Scope/Limitation

The scope is limited to network-level analysis, where detection is based solely on observable traffic features such as payload characteristics and request context. The study is conducted in a controlled experimental environment using a small set of custom-built extensions to simulate exfiltration behavior.

Some limitations apply. First, the evaluation covers only a limited number of malicious behaviors, primarily focused on data exfiltration, so it may not generalize more advanced and obfuscated attacks. Second, the dataset is generated in a controlled setting, which may not fully reflect the diversity and complexity of real-world traffic. Thirdly, the detection engine is limited to dynamic analysis, meaning the lack of static analysis can lead to less accuracy since the combination of the two is most effective. The detection engine proposed in this study is therefore meant as complementary, and it's not intended to be used completely on its own.

1.7 Target group

Cybersecurity analysts, enterprise network security administrators, and security operations center (SOC) personnel are the primary focus of this research, as they design/implement behavioral-based detection systems that protect organizational endpoints against potential

client side threats. In addition, this work has great relevance for browser extension developers interested in understanding modern security architectures such as “Manifest V3” and how they can abuse authorization models.

1.8 Outline

The following is how the thesis is organized:
Chapter 2 explains the methodology of our research, Chapter 3 gives the theoretical background of the research. Chapter 4 describes the implementation of the methodology. In Chapter 5 the results are presented, These are analyzed and further discussed in chapter 6 and 7. The thesis is then concluded with chapter 8 where conclusions and future work are mentioned.

2 Method

The main research method used in this study is Design Science Research Methodology (DSRM), since the project focuses on designing, implementing, and evaluating a detection artifact. In this case, the artifact is a proxy-based detection system intended to identify malicious browser extension traffic through structural analysis of HTTP requests.

To support the overall DSRM approach, the study also makes use of additional methods and techniques during the design and evaluation process. This includes constructing a labelled dataset through controlled traffic collection, where malicious browser extensions were developed specifically for testing and benign extensions were used to represent legitimate traffic. The evaluation of the detection artifact is then carried out using a Random Forest classifier together with Leave-One-Out Cross-Validation (LOOCV), in order to assess how well the selected network-level features can distinguish malicious extension traffic from benign traffic.

In practice, the study therefore combines artifact design with controlled empirical evaluation. DSRM provides the overall research structure, while network traffic analysis through a proxy, supervised machine learning, and cross-validation are used as concrete methods for implementing and testing the solution.

2.1 Research Project

This study is based on Design Science Research (DSR), as the purpose was to develop and evaluate a technical artefact rather than only analyse the problem itself. DSR is a well-established approach in information systems research for studies that aim to design artefacts to address identified problems while also generating knowledge through their evaluation, making it approptiate and a suitable methodological foundation for the proxy-based detection system developed in this thesis [5]

DSR consists of three connected cycles: the Relevance Cycle, the Design Cycle, and the Rigor Cycle.

The **Relevance Cycle** connects the research to a real-world problem and its stakeholders. In this study, the problem is that browser extensions are often granted broad permissions such as access to cookies, browsing history, and user input, while users and security systems have limited visibility into what data is actually being transmitted externally. Existing approaches often rely on browser-internal monitoring techniques, which advanced malicious extensions may be able to detect or bypass. Because of this, the project focuses on an external monitoring approach that operates outside the browser itself.

The stakeholders include end users, enterprise security teams, and browser platform developers. The threat model in this study includes three malicious behaviours: cookie exfiltration, keystroke logging, and browsing history leakage. These attack behaviours were reproduced during controlled traffic generation and used as the basis for evaluation.

The **Design Cycle** focuses on building, testing, and improving the artifact. In this project, the artifact is a proxy-based detection engine implemented as a `mitmproxy` addon. The system intercepts outgoing HTTP requests generated by browser extensions, extracts structural features from each request, and classifies the requests as either benign or malicious using a Random Forest classifier.

The dataset consists of traffic generated by ten browser extensions. The seven benign extensions were real-world extensions obtained from official browser extension repositories. The three malicious extensions were developed specifically for this study to implement representative data exfiltration behaviours, namely cookie theft, keystroke logging, and browsing history leakage. This provided complete control over the malicious behaviour and ensured reliable ground-truth labels during evaluation.

A major part of the design process in this study involved feature engineering. Instead of relying on things like destination blacklists or deep inspection of request content, the selected HTTP features focus on structural patterns in the traffic itself. Features such as payload size, delimiter density, and HTTP method were chosen because they can reflect how data is packaged and sent over the network, which makes them relevant for identifying possible exfiltration behaviour. In the detection system, these features are first used by the Random Forest model to assess individual HTTP requests. If an extension repeatedly generates requests that cross the anomaly threshold over time, it can then be classified as malicious. This approach also connects to earlier work such as Arcanum [1], which showed that network-level features can provide useful signals for identifying malicious extension behaviour.

Table 2.1 presents the extracted HTTP request features used by the detection engine.

Feature	Description	Relevance to Detection
length	Total byte length of the request body	Exfiltration payloads such as cookies or browsing history are often larger than normal requests.
num_equals	Number of "=" characters in the request body	Serialized key-value data commonly contains many equals signs.
num_semicolons	Number of ";" characters in the request body	Cookies and structured payloads often use semicolons as separators.
avg_substring_length	Average length of substrings separated by "&"	Long substrings may indicate encoded or serialized payloads.
num_query_params	Number of query parameters in the URL	Large parameter counts may indicate structured exfiltration traffic.
is_get	Binary flag indicating a GET request	Some exfiltration techniques append payloads directly to URLs.
is_post	Binary flag indicating a POST request	POST requests are commonly used for transmitting stolen data.
is_options	Binary flag indicating an OPTIONS request	Typically associated with CORS preflight requests rather than payload transmission.
is_trace	Binary flag indicating a TRACE request	TRACE requests are uncommon and may indicate unusual behaviour.

Table 2.1: Extracted HTTP Request Features

The Random Forest algorithm was selected because it works well with both binary and numerical features, handles smaller datasets reasonably well, and provides feature importance values that make the model easier to interpret.

The development process was iterative. The detection engine was continuously tested and refined throughout the project based on evaluation results and observed classification behaviour. Different feature combinations, classification thresholds, and traffic patterns were explored in order to improve the system's ability to distinguish malicious extension traffic from benign network activity.

The **Rigor Cycle** connects the project to existing research and scientific knowledge. Previous studies in browser extension security and network anomaly detection influenced both the feature selection process and the overall detection approach. The selected features were based on structural characteristics commonly found in serialized or exfiltrated HTTP payloads. In addition, the use of Random Forests, balanced class weights, and Leave-One-Out Cross-Validation (LOOCV) follows common practices in machine learning evaluation for smaller datasets.

Other research methods, such as case studies and action research, were considered but not selected. Since the goal of this project was to design and evaluate a technical artifact rather than study organisational behaviour or long-term processes, DSR was considered the most suitable research method.

2.2 Research methods

2.3 Network traffic analysis to detect malicious behaviour

Network traffic analysis was chosen as the main method for detecting malicious browser extension behaviour because many malicious actions, such as cookie exfiltration, history theft, or the transmission of captured user input, require communication with an external server. By analysing the traffic generated during extension execution, it becomes possible to observe whether an extension performs suspicious outbound communication that may indicate malicious behaviour. A proxy-based approach was used to capture this traffic in a controlled environment without modifying the browser itself. This approach is supported by previous research showing that network activity can be used to identify malicious browser extension behaviour and other forms of malware-related communication [6], [1]. Compared with static analysis, network traffic analysis makes it possible to examine actions that only become observable when the extension is actively interacting with browser data or remote endpoints.

2.4 Dataset

To examine whether malicious browser extension behaviour can be detected through network traffic analysis, this study developed a set of browser extensions to serve as benchmark artifacts. The aim was not to recreate complete real-world malicious extensions, but rather to implement selected malicious behaviours in a controlled and reproducible way. This made it possible to generate network traffic where the behaviour behind the traffic was known, which was important for evaluating whether malicious and benign extension activity could be distinguished. The decision to develop these extensions was motivated both by the need for experimental control and by the practical difficulty of collecting a large dataset of real malicious browser extensions. Previous research has shown that building such datasets can be a long and resource-intensive process; for example, one study collected thousands of benign and malicious Chrome extensions over a period of 14 months [1]. In contrast, creating benchmark extensions within the study made it possible to isolate and test specific behaviours, such as cookie exfiltration, browsing history retrieval, and keystroke capture, one at a time. This approach is also supported by earlier work that developed malicious browser extensions based on categories of malicious actions in order to study their behaviour in a controlled setting, , the extensions were designed to model common browser-extension abuse scenarios reported in prior work, namely cookie theft, keystroke capture, and browsing-history/page-content exfiltration. [7]. The benign extensions included in the study were selected from the Firefox Add-ons store.

For the cookie-exfiltration scenario, we first browsed websites that were likely to set cookies during the session so that the browser actually contained cookie data to steal. The extension then read ‘document.cookie‘ from the visited pages and, at regular intervals, sent the collected cookie values to the remote exfiltration server. This behaviour was treated as malicious because the extension was accessing cookie data that had nothing to do with its supposed functionality and transmitting it to an outside server without the user’s knowledge or any legitimate need in the page context. Rather than relying on plaintext inspection, the detection engine flags these exfiltration events by analysing structural irregularities in the HTTP request itself, such as unusually high delimiter density and abnormal payload lengths, which can arise when raw cookie data is serialised and transmitted in the request body

For the keylogging scenario, user input was manually entered into ordinary web form fields during the browsing session so that the extension had realistic keystroke data to capture. The extension attached event listeners to input and textarea elements and forwarded the typed values to the exfiltration server when focus left the field. This behaviour was treated as malicious because the captured keystrokes were transmitted to an external endpoint independently of the web page's intended functionality and without the user's knowledge or consent.

For the history and page-text leakage scenario, the browsing script intentionally visited a predefined set of pages before and during collection so that the extension could access non-empty browsing history and page content. The extension extracted either visible page text or page-related metadata and transmitted it to the exfiltration server as a JSON payload. This models privacy-invasive extensions that collect a user's browsing activity or viewed content for surveillance or profiling purposes.

2.5 Evaluation

Random Forest was selected as the main classification method because the study relies on structured traffic-derived features rather than raw packet data, making the problem well suited to a tree-based model. Random Forest has been widely applied in cybersecurity and network traffic classification because it can handle high-dimensional feature sets, model non-linear relationships, and remain robust even when features vary in importance or contain noise [8]. Previous work in malware and traffic analysis has also shown that Random Forest can achieve strong classification performance while still providing some interpretability through feature importance, which made it a suitable choice for this study [8].

Leave-One-Out Cross-Validation (LOOCV) was used to evaluate the model because the dataset in this study was relatively limited, and LOOCV makes it possible to train on almost the entire dataset in each iteration. This was considered useful in order to maximise the amount of training data while still obtaining a systematic estimate of model performance [9].

2.6 Reliability and Validity

In this study, reliability is supported using a controlled experimental environment where browser traffic is routed through a proxy and predefined extension behaviors generate known exfiltration events. By documenting the experimental setup, the engine logic (traffic collection process and detection rules etc.) other researchers should be able to replicate the experiment and obtain similar results. However, reliability may be affected by factors such as differences in browser version, network conditions, system configurations, etc. To reduce these risks, the environment is thoroughly documented and defined.

One potential threat to validity is that the experimental extensions simulate only a limited set of exfiltration techniques, which may not represent all real-world behaviors. To mitigate this, the most common exfiltration techniques are tested to make the experiments as comprehensive and wide as possible. Another concern is the small sample size of browser extensions. Relying on a limited number of extensions increases the risk that the ML model may focus on the specific beahaviour of those extensions rather than learning broader characteristics of data exfiltration. To address this, we will utilize the Leave One Out Cross-Validation (LOOCV) which is a proven evaluation technique to account for small datasets.

2.7 Ethical Considerations

This study does not involve human participants, surveys, or interviews; therefore, ethical concerns related to privacy and consent are non-existent. The research is conducted in a controlled experimental environment with malicious and benign browser extensions developed by us. The network traffic is synthetic, meaning it doesn't consist of actual sensitive data. The study focuses solely on technical system behavior, ensuring that no individuals are exposed to risk during the research process.

3 Theoretical Background

3.1 A Browser Extension Architecture and Privilege Models

The safety of contemporary web browsers depends on a multi-process structure intended to separate web content from the core operating system [10]. Nonetheless, browser extensions function within a distinct privileged level that connects the user's private information and the outside network [2].

3.1.1 The Manifest Blueprint and Permission Escalation

Each extension based on Chromium is regulated by a manifest.json file [11]. This document acts as the official statement of the extension's functionalities. Under the Manifest V3 standard, extensions are required to explicitly specify host permissions to engage with particular domains [11]. A major security issue occurs when extensions ask for "broad-host permissions" (e.g., $<all_urls>$), allowing them the ability to capture data from all websites the user accesses [10].

Table 3.2: Browser extension permissions and associated risks

Permission	**Technical Risk**	**Exfiltration Potential**
tabs	Access to URL history and titles.	High (Metadata sniffing)
cookies	Access to all session tokens.	Critical (Session Hijacking)
webRequest	Interception of all network traffic.	Critical (MITM)
<all_urls>	Injection of scripts into any site.	Absolute (Full DOM access)

3.1.2 Content Scripts and Isolation Environments

Extensions use "Content Scripts," which are JavaScript files inserted directly into the Document Object Model (DOM) of a website [10]. These scripts operate in an isolated environment to avoid clashes with the original JavaScript variables of the page, yet they have complete read/write access to the HTML nodes. This enables the covert retrieval of sensitive string information, like passwords and credit card details, prior to the web page encrypting them for transmission [10].

3.1.3 Background Service Workers

Functioning as the continuous conductor, the Service Worker oversees the extension's enduring logic [11]. It does not provide direct access to the DOM but acts as the main interface for network interactions. The Service Worker can trigger external POST requests to remote Command and Control (C2) servers by obtaining data from Content Scripts through internal message passing [10].

3.2 B Mechanics of Unauthorized Data Exfiltration

Data exfiltration concerning browser extensions is described as the illicit movement of confidential data from the client to an external entity [2]. This procedure generally adheres to a three-phase lifecycle: Collection, Staging, and Transmission.

3.2.1 DOM Scraping and Event Interception Logic

Malicious code frequently employs the addEventListener interface to track keyup or change events [10]. The subsequent reasoning demonstrates a standard exfiltration trigger:

- **Listener:** The extension adds a keydown event listener to the global window object [10].
- **Buffer:** Each key that is pressed gets recorded in a local string variable.
- **Threshold:** The buffer is packed once the string length surpasses 50 characters or a "submit" button is pressed.
- **Send:** The Service Worker is instructed to transfer the buffer to a distant endpoint.

3.2.2 Session Hijacking via Cookie Extraction

The chrome.cookies API allows extensions to access active session tokens [11]. An attacker can execute a "Session Replay" attack by exfiltrating a legitimate session cookie. This essentially circumvents multi-factor authentication (MFA) since the session is already set up and recognized as authenticated by the server.

3.3 C Proxy-Based Monitoring and SSL/TLS Inspection

Conventional network security tools frequently fail to detect modern browser-based threats due to the end-to-end encryption offered by HTTPS [12]. Analyzing network-layer exfiltration necessitates using an out-of-band, Man-in-the-Middle (MITM) proxy method that isolates the security monitor from the execution environment of the malware [12].

3.3.1 Architectural Integration of Mitmproxy

To achieve transparent observation of encrypted traffic, this research utilizes mitmproxy, an open-source, SSL/TLS-capable intercepting suite [12]. Mitmproxy sits architecturally between the client browser and the upstream internet, operating as a reverse or regular explicit proxy. Unlike hardware-based firewalls, mitmproxy provides a scriptable, event-driven engine capable of decoding and mutating application-layer protocols (HTTP/1, HTTP/2, and WebSockets) in real-time. Operating strictly out-of-band ensures that the monitoring engine remains hidden from sandbox-aware extension scripts executing inside the browser context.

3.3.2 SSL/TLS Termination and Upstream Certificate Sniffing

The proxy functions by capturing the TLS handshake between the web server and the browser [12]. This necessitates the setup of a specific, trusted Root Certificate Authority (CA) generated by mitmproxy and installed on the host operating system's trust store. The technical sequence of this interception is executed as follows:

1. **The Request:** The browser initiates a CONNECT request to an external domain (e.g., https://bank.com).
2. **Upstream Sniffing:** Mitmproxy intercepts the CONNECT packet, pauses the client side of the connection, and simultaneously initiates its own TLS handshake with the destination upstream server.

3. **Dynamic Generation:** Mitmproxy inspects the authentic server certificate to extract its Common Name (CN) and Subject Alternative Names (SANs). It uses these details to generate a spoofed, ephemeral certificate on the fly, signed by its trusted internal CA.

4. **The Interception:** The proxy returns this dynamically created certificate to the client browser [12].

5. **The Decryption:** Because the host trust store recognizes the signing CA, the browser completes the handshake. This splits the single HTTPS tunnel into two distinct cryptographic segments: Client-to-Proxy and Proxy-to-Server. The proxy decodes the traffic within its local memory space, offering a readable perspective of plaintext HTTP headers and body content.

6. **The Forwarding:** After the payload passes through the analysis script, the proxy forwards the data over the pre-established upstream secure link to the target server.

3.3.3 Event-Driven Inline Scripting Framework

The structural justification for selecting mitmproxy over traditional packet capture engines lies in its Event Hook Architecture [12]. Mitmproxy exposes stateful lifecycle hooks that allow external python-based heuristic modules to interact with specific stages of an HTTP flow:

- **requestheaders(flow):** Triggers immediately after HTTP request headers are read. This hook allows the heuristic engine to inspect target domains, request paths, and structural metadata before the request body is uploaded.

- **request(flow):** Triggers once the full HTTP request body has been received [12]. This event provides full access to the decoded payload, enabling the computation of Shannon entropy and pattern matching on potential data streams (such as exfiltrated cookies or keylogs).

- **response(flow):** Triggers once an upstream server response is completely read, allowing for the verification of successful Command and Control (C2) receipt acknowledgments.

By leveraging these programmatic abstractions, the proxy can execute complex statistical computations—such as calculating data entropy or evaluating temporal correlations—without causing fatal transmission latency within the active user session.

3.4 D Heuristic Detection and Entropy Analysis

Heuristic detection emphasizes unusual behaviors instead of recognized file hashes or IP blacklists [10].

3.4.1 Entropy Assessment

Grounded in Shannon's Information Theory, entropy gauges the unpredictability of a data sequence [13]. High-entropy payloads frequently suggest that the data is either compressed or encrypted. When a basic extension (like a "Clock" or "Weather" application) sends high-entropy data, it statistically suggests hidden data exfiltration.

3.4.2 Temporal Correlation

This entails examining the time difference between user activities and network occurrences [10]. A "Chain of Action" occurs when a DOM input event is promptly succeeded by a network transmission to a domain that has no recognized reputation.

4 Research Project – Implementation

The implementation consists of four main parts that work together: a set of malicious browser extensions representing different attack behaviours, a set of benign extensions used to generate normal traffic, a mitmproxy addon that intercepts and logs network requests, and a remote server that acts as the attacker-controlled exfiltration endpoint.

4.1 System Architecture

The system is built around a man-in-the-middle proxy placed between the browser and the network. The browser is configured to send all HTTP and HTTPS traffic through mitmproxy, which intercepts outgoing requests before they reach the destination server. The proxy addon checks the Origin header of each request. Requests originating from `moz-extension://` or `chrome-extension://` URLs are treated as browser extension traffic, while all other traffic is ignored.

For requests generated by extensions, the system extracts nine structural features from the HTTP request body and method. These features are then written as labelled rows into a CSV dataset file. The label, either malicious (1) or benign (0), is supplied as a command-line parameter when starting the proxy session, making it possible to separate different collection sessions cleanly.

4.2 Malicious Extension Design

Three malicious browser extensions were developed for the study, each representing a different type of browser-based data exfiltration attack.

The first extension performs cookie exfiltration. It uses browser permissions such as `tabs` and `activeTab` to access `document.cookie` from visited pages. A background script listens for page updates and extracts cookies whenever a page finishes loading. The collected cookie values are stored temporarily in memory and transmitted to the exfiltration server every 60 seconds using an HTTP POST request. The payload consists of newline-separated cookie strings gathered during the browsing session. This creates larger batched payloads that resemble real cookie-stealing behaviour.

The second extension acts as a keylogger. A content script is injected into pages and attaches event listeners to `<input>` and `<textarea>` elements. Whenever the user leaves an input field, the current value is captured and forwarded to the extension's background script using `chrome.runtime.sendMessage`. The background script then immediately sends the captured data to the exfiltration server as a JSON POST request containing the stolen value, the page URL, and a timestamp. Compared to the cookie exfiltration extension, this produces smaller but more frequent POST requests.

The third extension leaks browsing history and page text. It extracts the visible text content from visited pages, including the page title, URL, and either selected text or the full `document.body.innerText`. This information is serialized into JSON and transmitted through a POST request. Since the payload often contains large amounts of text and many URL-like strings, these requests tend to generate high values for features such as `body_length` and `num_semicolons`.

All three malicious extensions send their data to the same remote exfiltration endpoint.

4.3 Exfiltration Server

The exfiltration server is a lightweight Node.js application built using Express. It accepts POST requests at the `/exfil` endpoint and logs the received payloads to the console. The server supports both plain-text and JSON request bodies. Its only purpose is to provide a reachable external endpoint that the malicious extensions can communicate with during data collection.

4.4 Proxy Addon and Data Collection

The main data collection component is implemented as a mitmproxy addon written in Python. The addon registers a request hook that executes whenever a request passes through the proxy. The first step is checking whether the request originates from a browser extension by inspecting the Origin header for either the `moz-extension://` or `chrome-extension://` prefix. Requests not generated by extensions are discarded immediately.

For valid extension requests, the addon calls an `extract_features(flow)` function that computes the nine structural features described earlier in the methodology chapter. Together with the current timestamp, extension identifier, and session label, the extracted features are written to `dataset.csv` using Python's `csv.DictWriter`. The file is flushed after every write operation to reduce the risk of losing data if the collection session stops unexpectedly.

The addon is started from the command line using:

```
mitmproxy -s detection_engine/main.py --set label=1
```

where `label=1` indicates a malicious collection session and `label=0` indicates a benign session. The browser is configured to use mitmproxy as its HTTP/HTTPS proxy, and the mitmproxy CA certificate is installed in the browser trust store to allow TLS interception.

4.5 Data Collection Procedure

Data collection was carried out in separate sessions for each extension to ensure that the labels remained clean and consistent. For malicious extensions, the extension was loaded into Firefox as a temporary add-on, the proxy was started with `label=1`, and a predefined browsing script was executed. This involved visiting pages, interacting with forms, and allowing enough time for interval-based transmissions to occur.

The same browsing procedure was repeated for benign extensions using `label=0`. Using the same interaction flow for both malicious and benign extensions helped ensure that the resulting dataset reflected realistic browsing behaviour instead of artificially generated traffic spikes.

The final dataset contains 346 labelled HTTP requests: 45 malicious requests distributed across the three attack types and 301 benign requests generated by seven benign extensions. Each dataset row contains the timestamp, extension identifier, the nine extracted features, and the binary label.

4.6 Classifier Implementation

The Random Forest classifier and evaluation pipeline are implemented using sklearn library in python. The dataset is first loaded and the Boolean feature columns (`is_get`,

is_post, is_options, and is_trace) are converted into integer values. A RandomForestClassifier is then created using 100 estimators, a fixed random seed of 42, and class_weight="balanced" to compensate for the imbalance between malicious and benign samples.

Evaluation uses Leave-One-Out Cross-Validation (LOOCV) through scikit-learn's cross_val_predict.
For each iteration, one request is held out for testing while the remaining 345 requests are used for training. This process repeats for all 346 samples and produces out-of-sample predictions for the entire dataset. These predictions are then used to compute metrics such as precision, recall, F1-score and confusion matrix values. Probability estimates generated using predict_proba are additionally used for the ROC analysis.

To produce extension-level results, request predictions are grouped by extension identifier. The percentage of requests flagged as malicious is then calculated for each extension, and different thresholds are tested to generate the extension-level threshold analysis presented in the Results chapter.

5 Results

5.1 Dataset

The evaluation dataset comprises 346 labelled HTTP requests captured across ten browser extensions, of which three were malicious and seven were benign. The malicious extensions collectively produced 45 requests (13%), while the benign extensions produced 301 requests (87%), corresponding to approximately 43 requests per benign extension.

Each sample is represented by nine features extracted from the intercepted HTTP requests, as outlined in the methodology section.

5.2 Request-Level Classification

The Random Forest classifier was evaluated at the individual request level using Leave-One-Out Cross-Validation (LOOCV). Table 5.3 presents the confusion matrix values and classification metrics.

Metric	Value
Accuracy	97.69%
Precision	89.36%
Recall	93.33%
F1 Score	91.30%
True Positives	42
False Positives	5
True Negatives	296
False Negatives	3

Table 5.3: Request-Level Classification Results (LOOCV, $n = 346$)

Of the 45 malicious requests, 42 were correctly classified while 3 were missed. Of the 301 benign requests, 296 were correctly classified and 5 were incorrectly flagged as malicious.

5.3 Extension-Level Classification

Since the goal of the detection engine is to classify an extension as either benign or malicious, Request-level predictions were aggregated to produce a per-extension verdict. Two aggregation strategies were evaluated: a naive strategy in which an extension is flagged if any of its requests is classified as malicious, and a threshold-based strategy in which an extension is flagged only if the proportion of flagged requests meets or exceeds a threshold t.

Table 5.3 reports the extension-level classification results under different aggregation strategies.

Strategy	TP	FP	TN	FN	Precision	Recall	F1
Any request flagged	3	4	3	0	42.86%	100%	0.60
≥ 2 requests flagged	3	1	6	0	75.00%	100%	0.86
≥ 3 requests flagged	3	0	7	0	100%	100%	1.00
$\geq 5\%$ of requests flagged	3	0	7	0	100%	100%	1.00

Table 5.4: Extension-Level Results by Aggregation Strategy ($n = 10$ extensions)

Under the naive strategy (t = any), all three malicious extensions were detected, but four of the seven benign extensions were falsely flagged, resulting in a precision of 42.86% and an F1 score of 0.60. Applying a percentage threshold of $t \geq 5\%$ eliminated all false positives while retaining perfect recall within the evaluated dataset.

Table 5.4 presents the per-extension breakdown at the $t \geq 5\%$ threshold. Malicious extensions had between 84% and 100% of their requests flagged, while benign extensions had at most 5% flagged.

5.4 Feature Importances

Table 5.5 reports the Gini importance of each feature, computed by training the Random Forest classifier on the full dataset.

Feature	**Importance**
num_equals	0.4527
avg_substring_length	0.2357
body_length	0.1288
is_get	0.0653
num_query_params	0.0555
num_semicolons	0.0392
is_post	0.0228
is_options	0.0000
is_trace	0.0000

Table 5.5: Feature Importances

The results indicate that `num_equals` and `avg_substring_length` were the most influential features in the classification process, while HTTP method indicators such as `is_options` and `is_trace` contributed minimally to the model.

6 Analysis

6.1 Request-Level Performance

The classifier achieved an F1 score of 91.30%. This suggests that the extracted network features were generally effective at separating malicious exfiltration traffic from benign extension traffic.

The model produced 3 false negatives, meaning that some malicious requests were incorrectly classified as benign. These requests typically contained shorter or less structured payloads, making them harder to distinguish from normal traffic. In addition, 5 benign requests were falsely flagged as malicious. These cases mostly came from legitimate extensions sending structured POST requests that resembled exfiltration payloads in terms of size and formatting. This shows that some overlap exists between benign and malicious traffic when only structural network features are considered.

6.2 Feature Importance

The most important feature was `num_equals`, with an importance score of 0.4527. This feature counts the number of "=" characters in the request body and appears to capture the structure of cookie-like key-value data commonly found in exfiltration payloads.

Other influential features included `avg_substring_length` and `body_length`, suggesting that malicious requests often contained larger payloads and longer substrings than typical benign traffic.

The features `is_options` and `is_trace` had no measurable impact on the model. This is likely because none of the malicious extensions in the dataset used these HTTP methods. Although these features were not useful in this evaluation, they were retained to support future testing with a broader range of attack behaviours.

6.3 Extension-Level Classification and Threshold Analysis

When request-level predictions were aggregated to classify entire extensions, the aggregation strategy had a significant impact on the results. Using the naive approach, where an extension is flagged if any request is classified as malicious, resulted in perfect recall but also generated several false positives. Four of the seven benign extensions were incorrectly flagged, reducing precision considerably.

To reduce these false positives, a threshold-based strategy was introduced. By requiring at least 5% of an extension's requests to be flagged before classifying it as malicious, all benign extensions were correctly cleared while all malicious extensions remained detected. The results indicate that malicious extensions generated suspicious traffic more consistently, while benign extensions only produced isolated false positive requests.

7 Discussion

This chapter interprets the results in relation to the research questions and prior work. The study aimed to determine whether malicious browser extensions could be detected from their outgoing network traffic alone, without browser-internal monitoring or source code analysis. Overall, the approach proved feasible, though some important limitations emerged.

The malicious extensions were generally detected with high recall. For the keylogger and history-leakage extensions, every malicious request was correctly identified. This indicates that exfiltration traffic often has recognizable structural patterns, even when only the HTTP request is analysed rather than the payload content. Per-request detection is not enough. The main challenge was false positives: some benign extensions produced requests structurally similar to exfiltration, particularly larger POSTs or structured key-value data. Examining single requests in isolation is therefore unreliable. Aggregating to the extension level, flagging based on the percentage of suspicious requests,improved results significantly, since malicious extensions consistently generated a large proportion of suspicious traffic while benign ones produced only occasional false positives. This is a key finding: behavioural detection works better over time than on individual requests.

Random Forest importance showed request-structure features dominated such as, `body_length`, `avg_substring_length` and `num_equals`. This matches manual inspection: cookie exfiltration and history leakage produce long serialized payloads with many key-value pairs, raising both payload size and the number of equals signs. By contrast, `is_options` and `is_trace` had almost no impact, as TRACE requests were essentially absent and OPTIONS appeared mainly in normal CORS preflight traffic.

The findings are consistent with earlier behavioural-analysis research showing that network traffic can reveal suspicious behaviour without source code access [1]. But unlike approaches relying on browser APIs or deeper instrumentation, this study used a passive proxy, giving the extension less visibility into the monitoring and making evasion harder.

But limitations still apply, the dataset was small, ten extensions, only three malicious, and too limited to claim real-world generalisation. The attacks covered only a few types (cookie theft, keylogging, history leakage), excluding advanced techniques such as encrypted payloads, chunked exfiltration, or covert timing channels. The controlled environment and scripted interactions also reduced the variability of real browsing, which is noisier and would likely affect performance.

Even so, the study shows that passive network-level monitoring can detect several exfiltration behaviours without browser-internal APIs or source inspection. The performance figures should be read not as production-ready results, but as evidence that the approach is technically feasible and worth further exploration.

8 Conclusions and Future Work

This thesis investigated whether passive structural analysis of HTTP traffic at a network proxy can detect data exfiltration by malicious browser extensions without access to source code or payload inspection. The results show that the approach is feasible under controlled conditions. All three malicious extension types — cookie exfiltration, keystroke logging, and history leakage — were detected with 84–100% per-request recall using nine structural features derived only from HTTP request bodies and methods. The most useful features were `body_length`, `avg_substring_length`, `num_equals`, and `num_semicolons`, as they captured different structural patterns commonly found in exfiltration traffic.

At the same time, the results showed that per-request specificity was relatively low because of the class imbalance in the dataset. To make the system more practically useful, request-level predictions had to be aggregated at the extension level using a percentage threshold. One important finding from the study is therefore that the proxy works better as an evidence accumulator over time rather than as a per-request alarm system.

The study also helps address the gap of lacking a lightweight and passive detection layer operating at the network boundary. The approach was shown to be technically feasible, although the small dataset and controlled environment limit how far the results can be generalized to real-world deployment.

8.1 Future Work

Larger and more adversarial dataset. The biggest limitation of the study is the narrow dataset, consisting of three malicious and seven benign extensions. Future work should include traffic from a much broader range of extensions, including adversarial extensions designed to mimic benign traffic patterns through fragmentation, incremental transmission, or payload obfuscation. A larger dataset would also make it possible to use extension-grouped cross-validation methods that provide more reliable generalisation estimates than LOOCV on individual requests.

Non-HTTP channel coverage. The current system only monitors HTTP traffic and does not capture communication over protocols such as WebSockets or WebRTC. Since browser extensions can increasingly use these channels to bypass HTTP proxies entirely, extending detection to non-HTTP traffic would be necessary for a more realistic and production-ready system.

Real-world deployment study. The controlled experiment demonstrated proof-of-concept feasibility, but it cannot confirm whether the same feature patterns hold in real browsing environments with noisy background traffic and more diverse extension ecosystems. A longer real-world deployment study would provide the external validity that the current evaluation lacks.

References

[1] Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *Proceedings of the 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 2024. USENIX Association.

[2] Rezwana Pallan Karim, Mohan Dhawan, Vinod Ganapathy, and Chung-chieh Shan. An analysis of the Mozilla Jetpack extension framework. In *ECOOP 2012 – Object-Oriented Programming*, volume 7313 of *Lecture Notes in Computer Science*, pages 333–355. Springer, Berlin, Heidelberg, 2012.

[3] Hossain Shahriar, Komminist Weldemariam, Mohammad Zulkernine, and Thibaud Lutellier. Effective detection of vulnerable and malicious browser extensions. *Computers & Security*, 47:66–84, 2014.

[4] VmRay. Malware sandbox evasion techniques: All you need to know, April 2024. Accessed: 2026-05-21.

[5] Jan vom Brocke, Alan Hevner, and Alexander Maedche. Introduction to design science research. In *Design Science Research*, pages 1–13. Springer, Berlin, Germany, 2020.

[6] David Bekerman, Bracha Shapira, Lior Rokach, and Asaf Bar. Unknown malware detection using network traffic classification. In *2015 IEEE Conference on Communications and Network Security (CNS)*, pages 134–142, Florence, Italy, 2015. IEEE.

[7] Shreya Singh, Gaurav Varshney, Tarun Kumar Singh, Vidhi Mishra, and Khushi Verma. A study on malicious browser extensions in 2025. *arXiv preprint arXiv:2503.04292*, 2025.

[8] Michael Austin. Iot malicious traffic classification using machine learning. Graduate thesis / problem report, West Virginia University, 2021.

[9] G.C. Cawley. Leave-one-out cross-validation based model selection criteria for weighted ls-svms. In *The 2006 IEEE International Joint Conference on Neural Network Proceedings*, pages 1661–1668, 2006.

[10] Adam Barth, Adrienne Porter Felt, Prateek Saxena, and Aaron Boodman. An evaluation of the Google Chrome extension security architecture. In *Proceedings of the 21st USENIX Security Symposium*, pages 321–334. USENIX Association, 2012.

[11] Google Developer Group. Chrome extensions reference: chrome.declarativenetrequest API platform rules. `https://developer.chrome.com/docs/extensions/reference/api/declarativeNetRequest`, 2026. Accessed: 2026-05-21.

[12] Aldo Cortesi, Maximilian Hils, and Thomas Knaeble. Mitmproxy: An interactive SSL/TLS-capable intercepting HTTP proxy architecture. `https://docs.mitmproxy.org/`, 2026. Accessed: 2026-05-21.

[13] Claude Elwood Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

AI Statement

In the preparation of this work, we used AI as a support tool mainly for programming, language improvement and structure. The tool was used to help with coding but also give the text a more academic sound. It was also helpful for brainstorming how to organise sections and how to present technical explanations in a clearer way. The specific tools used were ChatGPT(openAI), Google Gemini 3.5 and Claude Code.

The purpose in using AI was for it to act as a writing assistant while also boosting productivity. As an example it helped with generating tables with results based on the provided information which saved time in getting the details done.

Reflecting on its use, we found AI valuable for improving presentation and communication, especially in expressing ideas more effectively and academically. At the same time, we were careful not to rely on it uncritically. We experienced AI-generated suggestions to sometimes be too inaccurate or not fully conveying meaning based on the requirements. Because of this, we made sure to review and evaluate all suggestions before including them in the final submission.